SOFT COMPUTING
GENETIC ALGORITHMS

Dr.Ahmed Anter
Associate Professor,
CSIT, E-JUST.
Spring, 2026

INTRODUCTION

After scientists became disillusioned with classical and neo-classical attempts at modeling intelligence, they looked in other directions.

Two prominent fields arose, connectionism (neural networking, parallel processing) and evolutionary computing.

It is the latter that this essay deals with - genetic algorithms and genetic programming.

GENETIC ALGORITHMS

"Genetic Algorithms are good at taking large, potentially huge search spaces and navigating them, looking for optimal combinations of things, solutions you might not otherwise find in a lifetime."

- Salvatore Mangano

Computer Design, May 1995

THE GENETIC ALGORITHM

Directed search algorithms based on the mechanics of biological evolution.

Developed by John Holland, University of Michigan (1970's)

- To understand the adaptive processes of natural systems.
- To design artificial systems software that retains the robustness of natural systems.

THE GENETIC ALGORITHM (CONT.)

Provide efficient, effective techniques for optimization and machine learning applications.

Widely-used today in business, scientific and engineering problems.

WHAT IS GA

A **genetic algorithm** (or **GA**) is a search technique used in computing to find true or approximate solutions to optimization and search problems.

Genetic algorithms are categorized as global search heuristics.

Genetic algorithms are a particular class of evolutionary algorithms that use techniques inspired by evolutionary biology such as inheritance, mutation, selection, and crossover (also called recombination).

WHAT IS GA

Genetic algorithms are implemented as a computer simulation in which a population of abstract representations (called chromosomes or the genotype or the genome) of candidate solutions (called individuals, or phenotypes) to an optimization problem evolves toward better solutions.

Traditionally, solutions are represented in binary as strings of 0s and 1s, but other encodings are also possible.

WHAT IS GA

The evolution usually starts from a population of randomly generated individuals and happens in generations.

In each generation, the fitness of every individual in the population is evaluated, multiple individuals are selected from the current population (based on their fitness), and modified (recombined and possibly mutated) to form a new population.

WHAT IS GA

The new population is then used in the next iteration of the algorithm.

Commonly, the algorithm terminates when either a maximum number of generations has been produced, or a satisfactory fitness level has been reached for the population.

If the algorithm has terminated due to a maximum number of generations, a satisfactory solution may or may not have been reached.

GA KEY TERMS

Individual - Any possible solution

Population - Group of all *individuals*

Search Space - All possible solutions to the problem

Chromosome - Blueprint for an *individual*

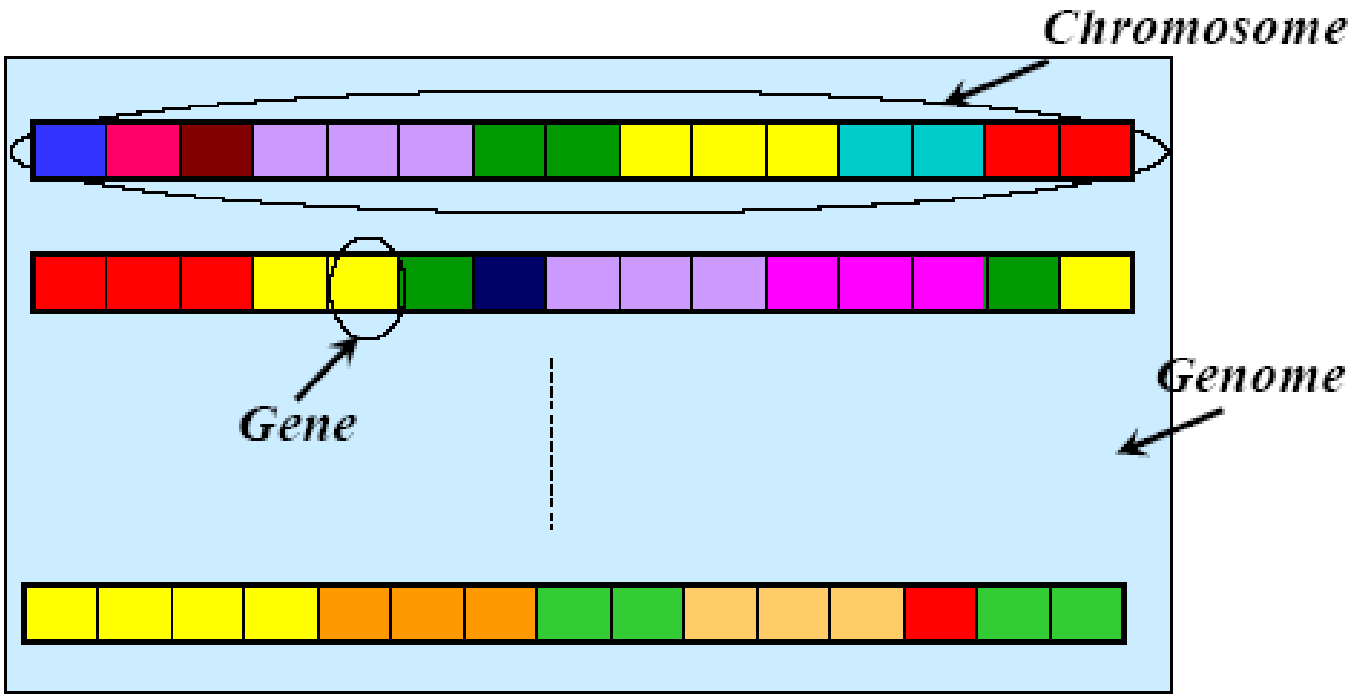
Trait - Possible aspect (*features*) of an *individual*

Allele - Possible settings of trait (black, blond, etc.)

Locus - The position of a *gene* on the *chromosome*

Genome - Collection of all *chromosomes* for an *individual*

CHROMOSOME, GENES AND GENOMES



GENOTYPE AND PHENOTYPE

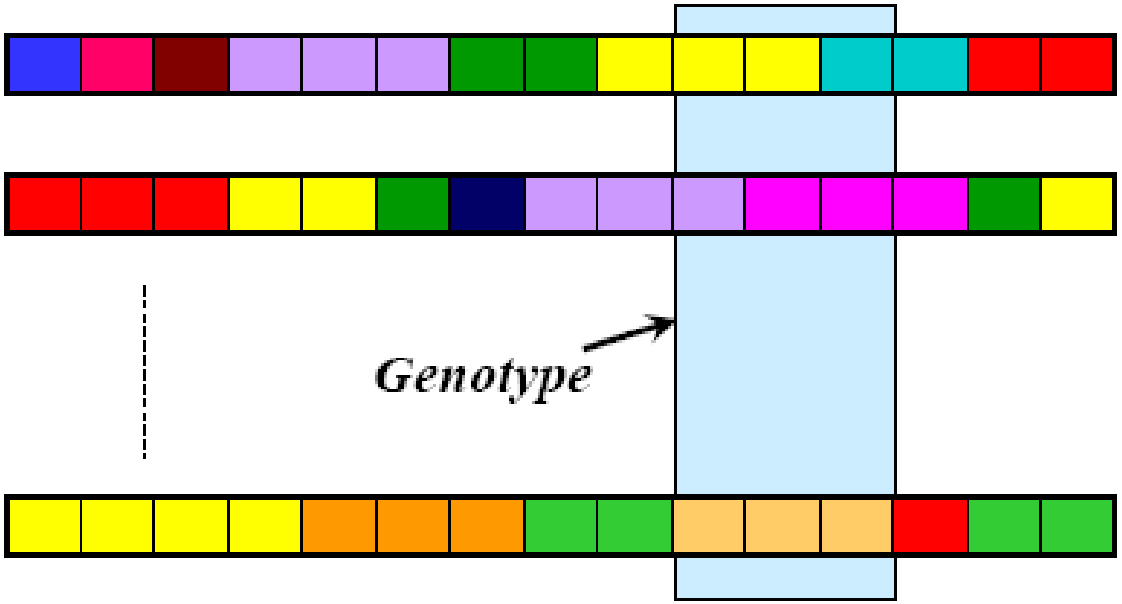
Genotype:

– Particular set of genes in a genome

Phenotype:

– Physical characteristic of the genotype (smart, beautiful, healthy, etc.)

GENOTYPE AND PHENOTYPE



GA REQUIREMENTS

A typical genetic algorithm requires two things to be defined:

- a genetic representation of the solution domain, and
- a fitness function to evaluate the solution domain.

A standard representation of the solution is as an array of bits. Arrays of other types and structures can be used in essentially the same way.

The main property that makes these genetic representations convenient is that their parts are easily **aligned** due to their fixed size, that facilitates simple crossover operation.

Variable length representations may also be used, but crossover implementation is more complex in this case.

REPRESENTATION

Chromosomes could be:

- Bit strings (0101 ... 1100)
- Real numbers (43.2 -33.1 ... 0.0 89.2)
- Permutations of element (E11 E3 E7 ... E1 E15)
- Lists of rules (R1 R2 R3 ... R22 R23)
- Program elements (genetic programming)
- ... any data structure ...

GA REQUIREMENTS

The fitness function is defined over the genetic representation and measures the *quality* of the represented solution.

The fitness function is always problem dependent.

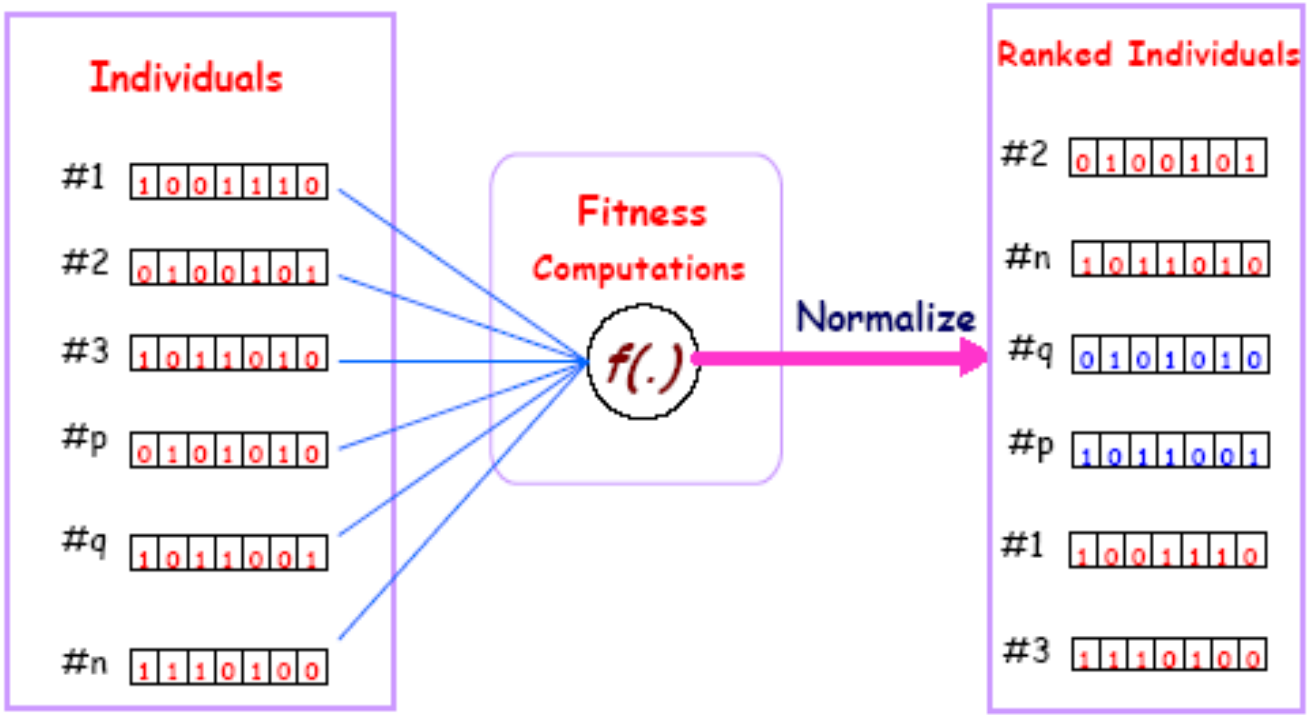
For instance, in the knapsack problem we want to maximize the total value of objects that we can put in a knapsack of some fixed capacity.

A representation of a solution might be an array of bits, where each bit represents a different object, and the value of the bit (0 or 1) represents whether or not the object is in the knapsack.

Not every such representation is valid, as the size of objects may exceed the capacity of the knapsack.

The *fitness* of the solution is the sum of values of all objects in the knapsack if the representation is valid, or 0 otherwise. In some problems, it is hard or even impossible to define the fitness expression; in these cases, interactive genetic algorithms are used.

A FITNESS FUNCTION



COMPONENTS OF A GA

A problem to solve, and ...

- Encoding technique (*gene, chromosome*)
- Initialization procedure (*creation*)
- Evaluation function (*environment*)
- Selection of parents (*reproduction*)
- Genetic operators (*mutation, recombination*)
- Parameter settings (*practice and art*)

BASICS OF GA

The most common type of genetic algorithm works like this:

- a population is created with a group of individuals created randomly.
- The individuals in the population are then evaluated.
- The evaluation function is provided by the programmer and gives the individuals a score based on how well they perform at the given task.
- Two individuals are then selected based on their fitness, the higher the fitness, the higher the chance of being selected.
- These individuals then "reproduce" to create one or more offspring, after which the offspring are mutated randomly.
- This continues until a suitable solution has been found or a certain number of generations have passed, depending on the needs of the programmer.

GENERAL ALGORITHM FOR GA

Initialization

- Initially many individual solutions are randomly generated to form an initial population. The population size depends on the nature of the problem, but typically contains several hundreds or thousands of possible solutions.
- Traditionally, the population is generated randomly, covering the entire range of possible solutions (the *search space*).
- Occasionally, the solutions may be "seeded" in areas where optimal solutions are likely to be found.

GENERAL ALGORITHM FOR GA

Selection

- During each successive generation, a proportion of the existing population is selected to a new generation.
- Individual solutions are selected through a *fitness-based* process, where fitter solutions (as measured by a fitness function) are typically more likely to be selected.
- Certain selection methods rate the fitness of each solution and preferentially select the best solutions. Other methods rate only a random sample of the population, as this process may be very time-consuming.
- Most functions are stochastic and designed so that a small proportion of less fit solutions are selected. This helps keep the diversity of the population large, preventing premature convergence on poor solutions. Popular and well-studied selection methods include **roulette wheel selection** and **tournament selection**.

GENERAL ALGORITHM FOR GA

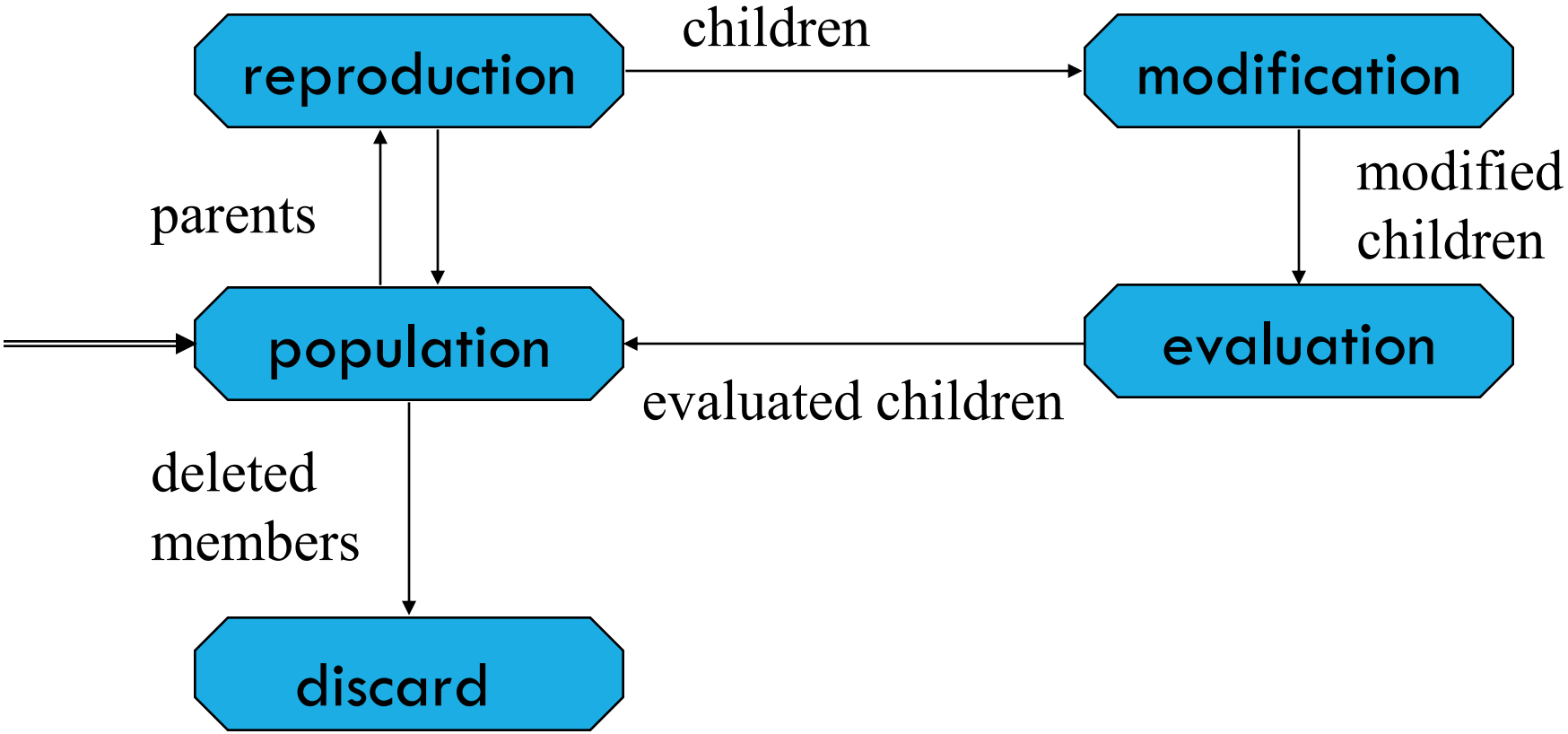
In roulette wheel selection, individuals are given a probability of being selected that is directly proportionate to their fitness.

Two individuals are then chosen randomly based on these probabilities and produce offspring.

SIMPLE GENETIC ALGORITHM

```
{
 initialize population;
 evaluate population;
 while TerminationCriteriaNotSatisfied
 {
   select parents for reproduction;
   perform recombination and mutation;
   evaluate population;
 }
}
```

THE GA CYCLE OF REPRODUCTION



POPULATION

Chromosomes could be:

- Bit strings (0101 … 1100)
- Real numbers (43.2 -33.1 … 0.0 89.2)
- Permutations of element (E11 E3 E7 … E1 E15)
- Lists of rules (R1 R2 R3 … R22 R23)
- Program elements (genetic programming)
- … any data structure …

GENERAL ALGORITHM FOR GA

Reproduction

- The next step is to generate a second generation population of solutions from those selected through genetic operators:
- Crossover (also called recombination), and/or mutation.
- For each new solution to be produced, a pair of "parent" solutions is selected for breeding from the pool selected previously.
- By producing a "child" solution using the above methods of crossover and mutation, a new solution is created which typically shares many of the characteristics of its "parents". New parents are selected for each child, and the process continues until a new population of solutions of appropriate size is generated.

GENERAL ALGORITHM FOR GA

- These processes ultimately result in the next generation population of chromosomes that is different from the initial generation.

- Generally, the average fitness will have increased by this procedure for the population, since only the best organisms from the first generation are selected for breeding, along with a small proportion of less fit solutions, for reasons already mentioned above.

CROSSOVER

- The most common type is **single point crossover**. In single point crossover, you choose a locus at which you swap the remaining alleles from on parent to the other. This is complex and is best understood visually.

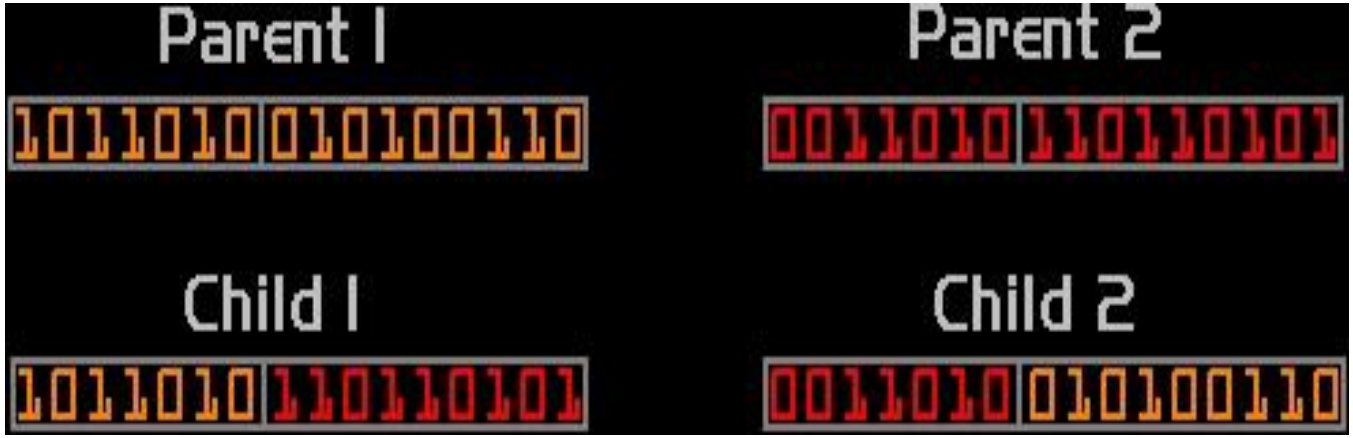
  As you can see, the children take one section of the chromosome from each parent.

  The point at which the chromosome is broken depends on the randomly selected crossover point.

  This particular method is called single point crossover because only one crossover point exists. Sometimes only child 1 or child 2 is created, but oftentimes both offspring are created and put into the new population.

  Crossover does not always occur, however. Sometimes, based on a set probability, no crossover occurs and the parents are copied directly to the new population. The probability of crossover occurring is usually 60% to 70%.

CROSSOVER (RECOMBINATION)



MUTATION

- After selection and crossover, you now have a new population full of individuals.
- Some are directly copied, and others are produced by crossover.
- In order to ensure that the individuals are not all exactly the same, you allow for a small chance of mutation.
- You loop through all the alleles of all the individuals, and if that allele is selected for mutation, you can either change it by a small amount or replace it with a new value.
- Mutation is fairly simple. You just change the selected alleles based on what you feel is necessary and move on. Mutation is, however, vital to ensuring genetic diversity within the population.

MUTATION

GENERAL ALGORITHM FOR GA

Termination

- This generational process is repeated until a termination condition has been reached.
- Common terminating conditions are:
  - A solution is found that satisfies minimum criteria
  - Fixed number of generations reached
  - Allocated budget (computation time/money) reached
  - The highest ranking solution's fitness is reaching or has reached a plateau such that successive iterations no longer produce better results
  - Manual inspection
  - Any Combinations of the above

GA PSEUDO-CODE

Choose initial population

Evaluate the fitness of each individual in the population

Repeat

Select best-ranking individuals to reproduce

Breed new generation through crossover and mutation (genetic operations) and give birth to offspring

Evaluate the individual fitnesses of the offspring

Replace worst ranked part of population with offspring

Until <terminating condition>

Example 1: A Very Simple Example

- Generate a bit string for the decimal number 31.
- Traditional Technique:
  - Start by 00000 till reaching 11111
- GA Steps:
  1. Build an initial population of random individuals (chromosomes or initial solutions)
  2. Evaluate fitness of individuals
  3. Select some individuals to
     - Apply Crossover and Mutation
  4. Build next generation
  5. Go to step 2.

Example 1: GA Step 1

31

0 1

r = 0.7 > 0.5

0 → 1

- Build an initial population of <u>*random*</u> individuals

pop_sz = 4

G_0 | 00100 | 10100 | 01011 | 00101 |

- Generation 0

Example 1: GA Step 2

- Evaluate *fitness* of individuals

	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101
	4	20	11	5

- Fitness can be calculated as the decimal value of each bit string

Example 1: GA Step 3

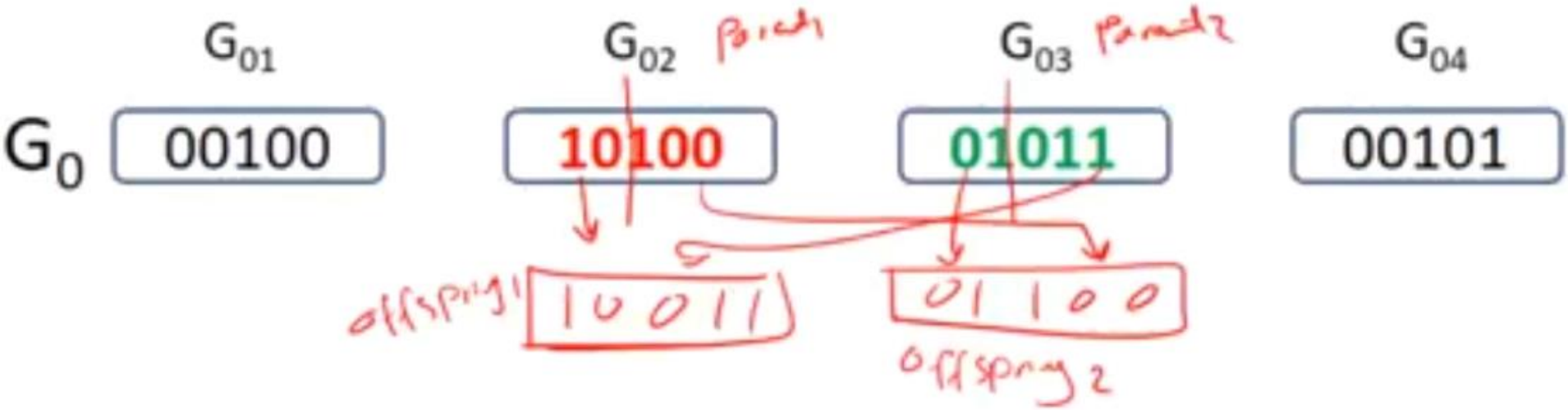
- Select some individuals to apply *crossover* and mutation

	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101
	4	20	11	5

- Individual with good fitness should be selected for crossover and mutation.

Example 1: GA Step 3 – Trial 1

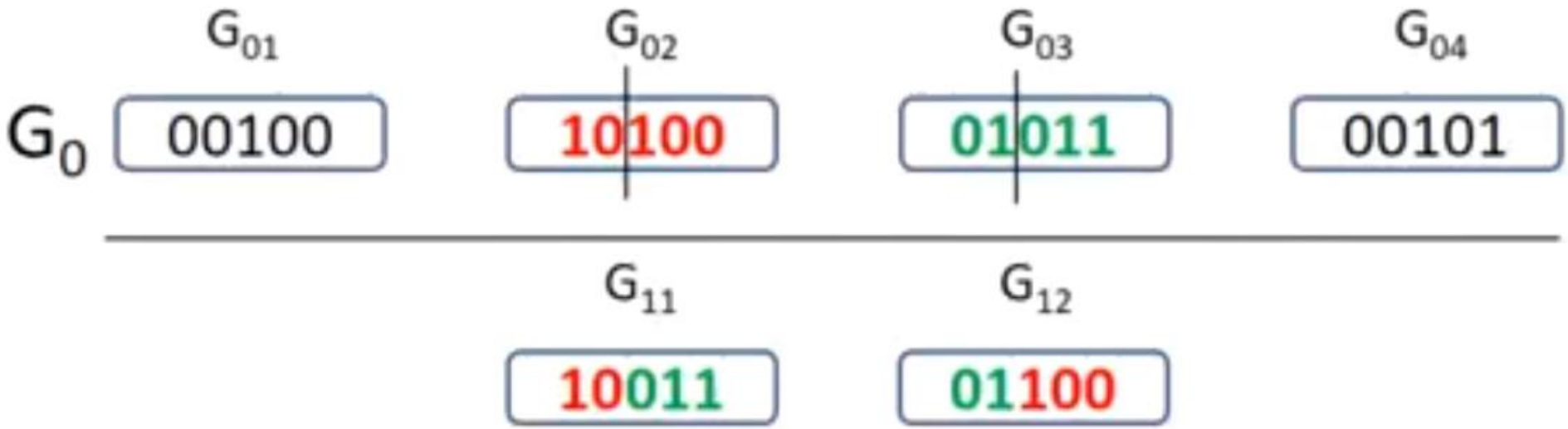
- Select some individuals to apply *crossover* and mutation



- Single point crossover after 2 bits (Genes)

Example 1: GA Step 3 – Trial 1

- Select some individuals to apply *crossover* and mutation



Fitness?

Example 1: GA Step 3 – Trial 1

- Select some individuals to apply *crossover* and mutation

		20	11	
	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101
		G_{11}	G_{12}	
		10011	01100	
		19	12	

Example 1: GA Step 3 – Trial 2

- Select some individuals to apply *crossover* and mutation

	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	**10100**	**01011**	00101

- Single point crossover after 3 bits

Example 1: GA Step 3 – Trial 2

- Select some individuals to apply *crossover* and mutation

		20	11	
	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101
		G_{11}	G_{12}	
		10111	01000	
		23	8	

Fitness?

Example 1: GA Step 3 – Trial 2

- Select some individuals to apply crossover and ***mutation***

		20	11	
	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101
	G_{14}	G_{11}	G_{12}	G_{13}
	00100	10111	01000	00101
	4	23	8	5

- How to apply a mutation over some individual?
  - For bit strings, and according to the mutation probability of your implementation, one bit might be flipped!

Example 1: GA Step 3 – Trial 2

- Select some individuals to apply crossover and ***mutation***

		20	11	
	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101
	G_{13}	G_{11}	G_{12}	G_{14}
	00100	10111	01000	00101
	4	23	8	5

Example 1: GA Step 3 – Trial 2

- Select some individuals to apply crossover and ***mutation***

		20	11	
	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101
	G_{13}	G_{11}	G_{12}	G_{14}
G_1	00100	11111	01000	00101
	4	31	8	5

OUR OBJECTIVE!

Example 1: GA Step 3 – Trial 3

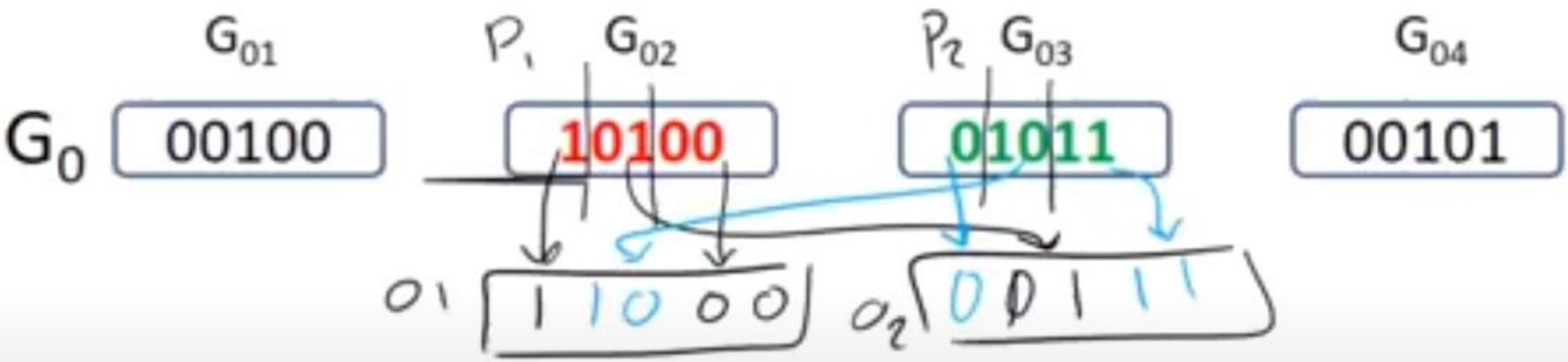
- Select some individuals to apply crossover and mutation

	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101

- Applying multiple point crossover!

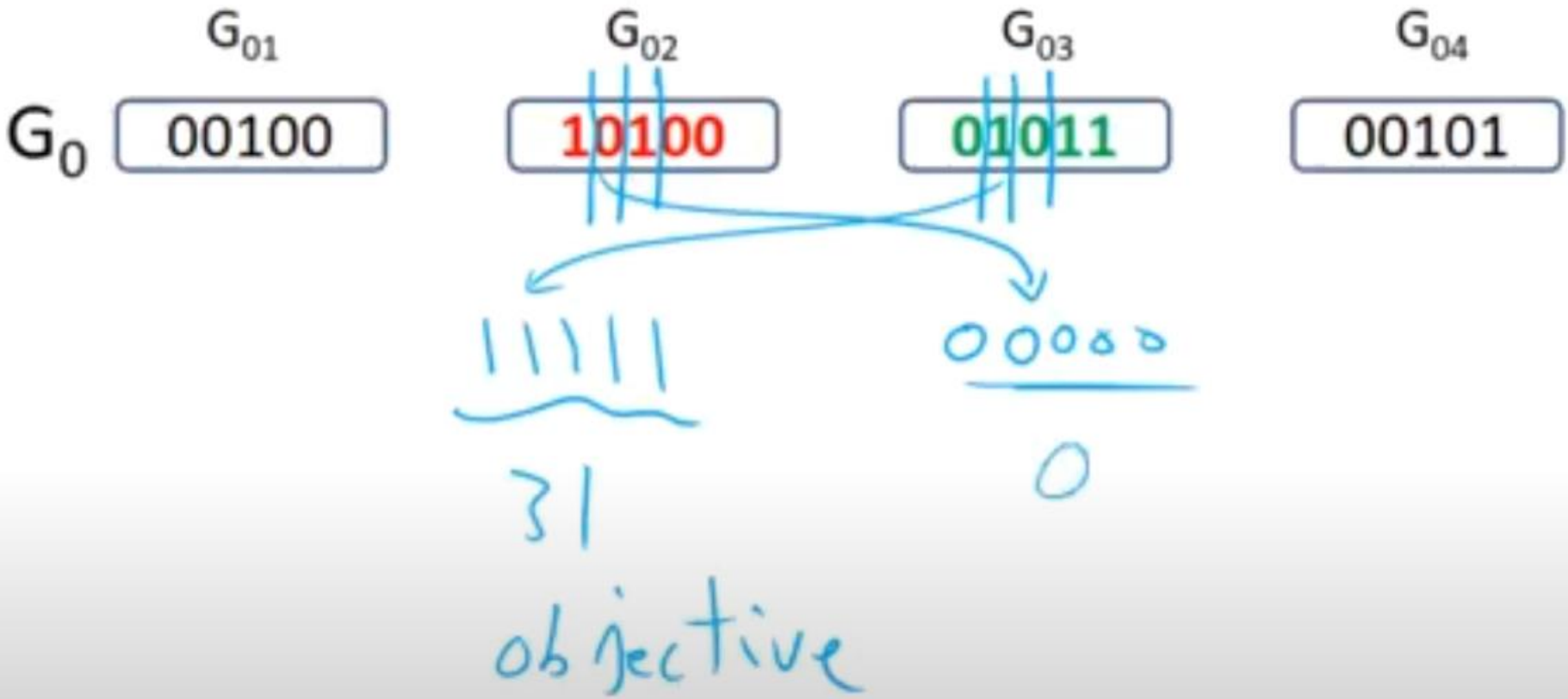
Example 1: GA Step 3 – Trial 3

- Select some individuals to apply crossover and mutation



- Applying multiple point crossover!

2 point " 1, 3


G01
G02
G03
G04
G0
00100
10100
01011
00101
11111
00000
31
0
objective

Example 1: GA Step 3 – Trial 4

- Select **ALL** individuals to apply *crossover* and mutation

	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101
	4	20	11	5
Crossover	G_{11}	G_{13}	G_{12}	G_{14}
	00011	10101	01100	00100
	3	21	12	4
Mutation		11101		00110
		21→29		4→6

Example 1: GA Step 4

- Build next generation .. Mixing

	G_{02}	G_{03}	G_{11}	G_{13}
G_1	10100	01011	00011	11101
	20	11	3	29
Crossover	G_{02}	G_{03}	G_{22}	G_{22}
	10100	01011	00001	11111
	20	11	1	31
Mutation	G_{02}	G_{03}	G_{21}	G_{22}
	10100	11011	00000	11111
	20	27	0	31

OUR OBJECTIVE!

Does it really cost one iteration?

- Real world problems are not that easy.
- You could have:
  - Hundreds of individuals (chromosomes)
  - Huge search space
- You might perform thousands of iterations (have thousands of generations) before reaching your objective!

How do you encode a solution?

- Obviously this depends on the **problem**!
- GA's *often* encode solutions as fixed length "*bitstrings*" (e.g. 101110, 111111, 000101)
- Sometimes, your solutions (chromosomes) will be used in their original format (*floating point numbers* or *alphabetical strings*)

Why does crossover work?

- The idea is that crossover preserves "good bits" from different parents, combining them to produce better solutions
- The fitness may be improved or get worst after mutation
- A good scheme would therefore try to preserve "good bits" during crossover and mutation

Which problems can GAs solve?

- The solution can be encoded.
- Each gene represents some aspect of the proposed solution to the problem (the solution can be decomposed into smaller parts)
- The ability to "test" any solution and get a "score" indicating how "good" that solution is (**Fitness**).

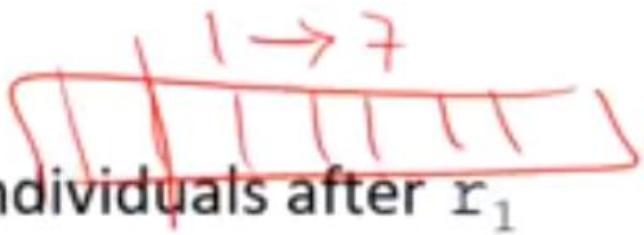
Implementing Randomization in GAs

Canonical Genetic Algorithm

```
Canonical_Genetic_Algorithm()
{
   Initialize the Population = G0; // population size is constant …

   For(i=1 to Max_Generations) // Continue Evolution …
   {
        Evaluate Fitness of Individuals of Gi-1; // By Objective Function

        Select Parents for Reproduction; // Roulette Wheel?

        Crossover; // According to probability of crossover…

        Mutation;  // According to probability of mutation

        Replacement;  // Replacing old generation Gi-1 with new generation Gi
   }
}
```

Crossover Probabilities

- P_c fixed parameter in GA [0.4 → 0.7]
  - Probability that crossover will occur between two selected chromosomes
- Generate random number $r_1 \in [1, L-1]$
  - Where L = length of chromosome
- Crossover point = r_1
  - Crossover will occur between two individuals after r_1 genes
- Generate random number $r_2 \in [0, 1]$
  - if $r_2 \leq P_c$ then perform crossover
  - else no crossover occur …
    - Offspring1 = Parent1
    - Offspring2 = Parent2



Crossover Probabilities

1 → 4

- Assume $P_c = 0.7$

$r_1 = 2$

	G_{01}	G_{02}	G_{03}	G_{04}
G_0	00100	10100	01011	00101

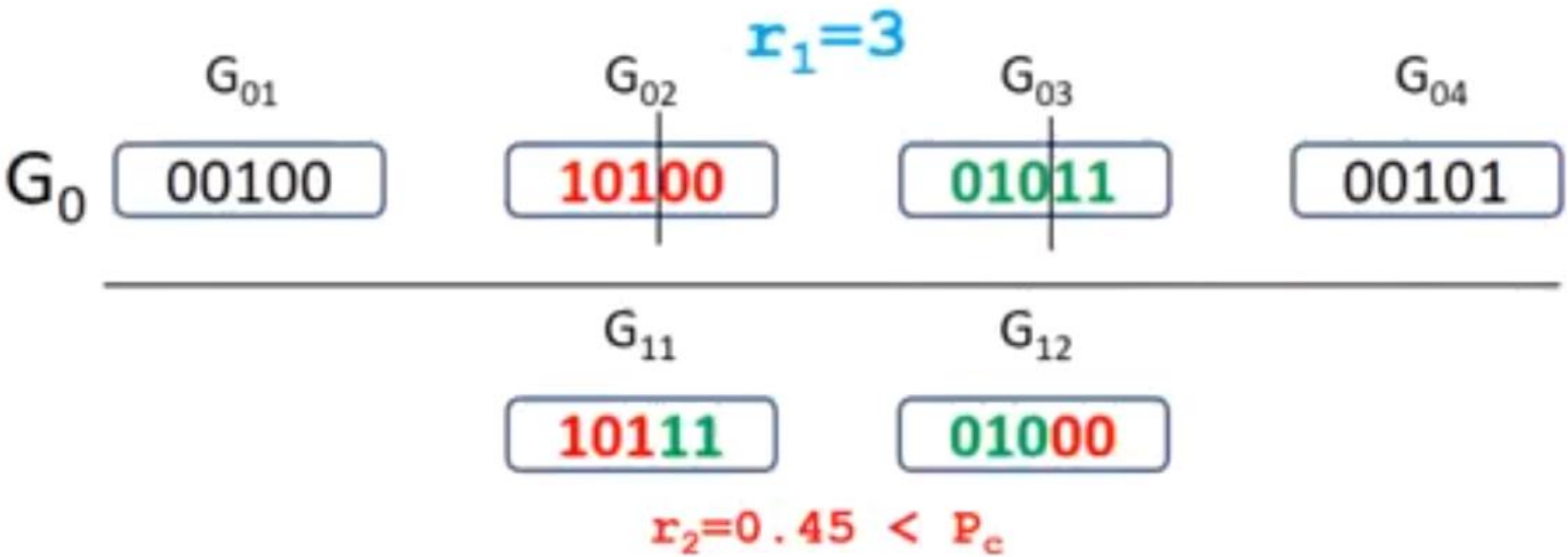
G_{11}	G_{12}
10011	01100

$r_2 = 0.6 < P_c$

- Single point crossover after 2 bits (Genes)

Crossover Probabilities

- Assume $P_c = 0.7$



- Single point crossover after 3 bits

Mutation Probabilities

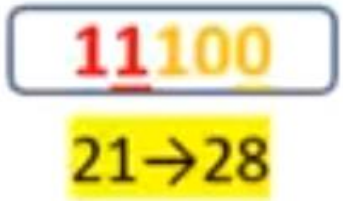
- P_m fixed parameter in GA [0.001 → 0.1]
  - Probability that mutation will occur for a gene/bit in some chromosomes
- Chromosome = bits[1…L]

```
for(i=1 to L)
{
    Generate Random number r_i ∈ [0, 1]
    if(r_i ≤ P_m)
        flip bit[i]
    elseif(r_i > pm)
        no change to bit[i]
}
```

Mutation Probabilities

- $P_m = 0.1$

r_i	Value
r_1	0.2
r_2	0.01
r_3	0.5
r_4	0.11
r_5	0.1



Example : $f(x) = x^2$

- **Finding the maximum of a function:**
  - $f(x) = x^2$
  - Range [0, 63]
  - Binary representation: string length 6 → 64 numbers (0-63)

genotype	0	0	0	1	0	1
mapping			...	2^2	2^1	2^0
			...	4	2	1
phenotype	... 1*4+0*2+1*1 = 5					
fitness	25					= f(x)

$f(x) = x^2$

Initial Random Population

pop. sz = 5

	binary	value x	fitness x^2
String 1	000110 ✓	6	36
String 2	000011 ✓	3	9
String 3	001010 ✓	10	100
String 4	010101 ✓	21	441
String 5	000001 ✓	1	1

$f(x) = x^2$

Selection

	binary	value	fitness
String 1	000110	6	36
String 2	000011	3	9
String 3	001010	10	100
String 4	010101	21	441
String 5	000001	1	1

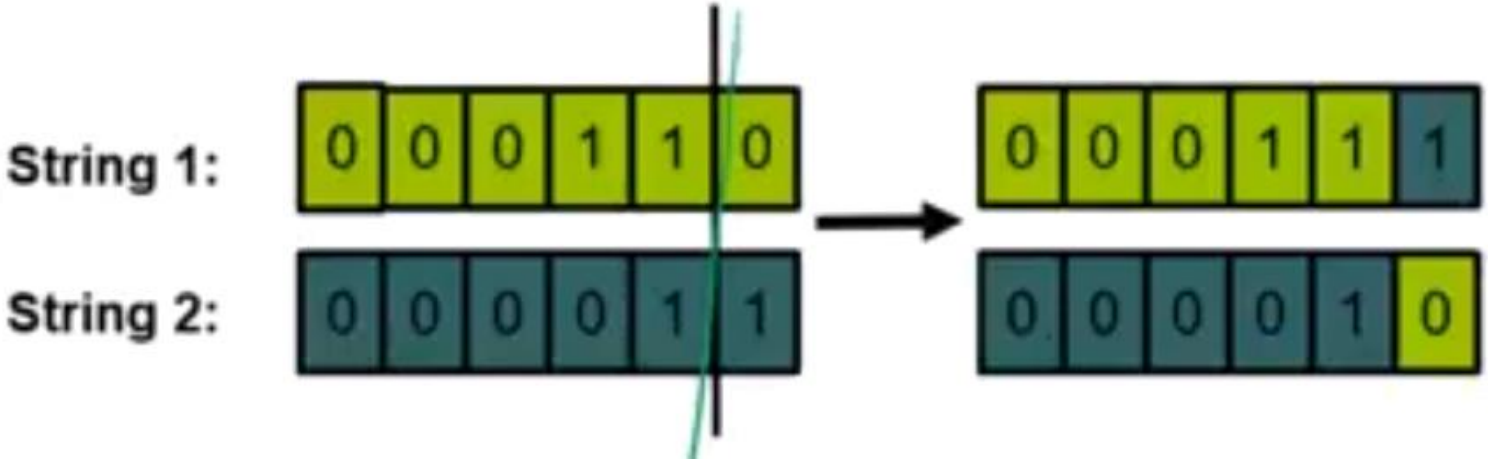
- Worst one can be removed

$f(x) = x^2$

Recombination

- Parents and x-position randomly selected

	partner	x-position
String 1	**String 2**	**5** r_1
String 3	String 4	3 r_1

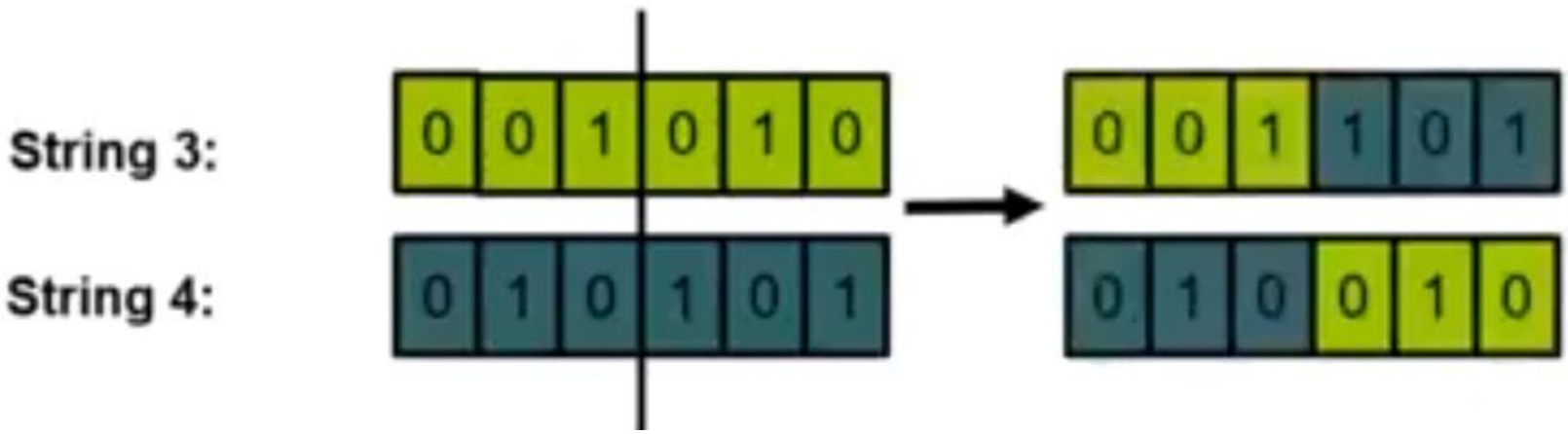


$f(x) = x^2$

Recombination

- Parents and x-position randomly selected

	partner	x-position
String 1	String 2	5
String 3	**String 4**	**3**



$f(x) = x^2$

Mutation

- bit-flip:
  - Offspring-String 1: 000111 (7) → 010111 (23)
  - String 5: 010101 (21) → 010001 (17)

$f(x) = x^2$ New Generation

- All individuals in the parent population are replaced by offspring in the new generation full
  - (generations are *discrete*!)
- **New population (Offspring):**

	binary	value	fitness
String 1	010111	23	529
String 2	000010	2	4
String 3	001101	13	169
String 4	010010	18	324
String 5	010001	17	289

$f(x) = x^2$

When to stop?

- Iterate until termination condition reached, e.g.:
  - Best fitness < optimal solution
    Accuracy > 95%
  - Number of generations
  - No New Chromosomes
  - No improvements after a number of generations

Drilling for Oil Example

- Imagine you had to drill for oil somewhere along a single 1km desert road
- Problem: choose the best place on the road that produces the most oil per day
- We could represent each solution as a position on the road

- Say, a whole number between [0..1000]

Digging for Oil

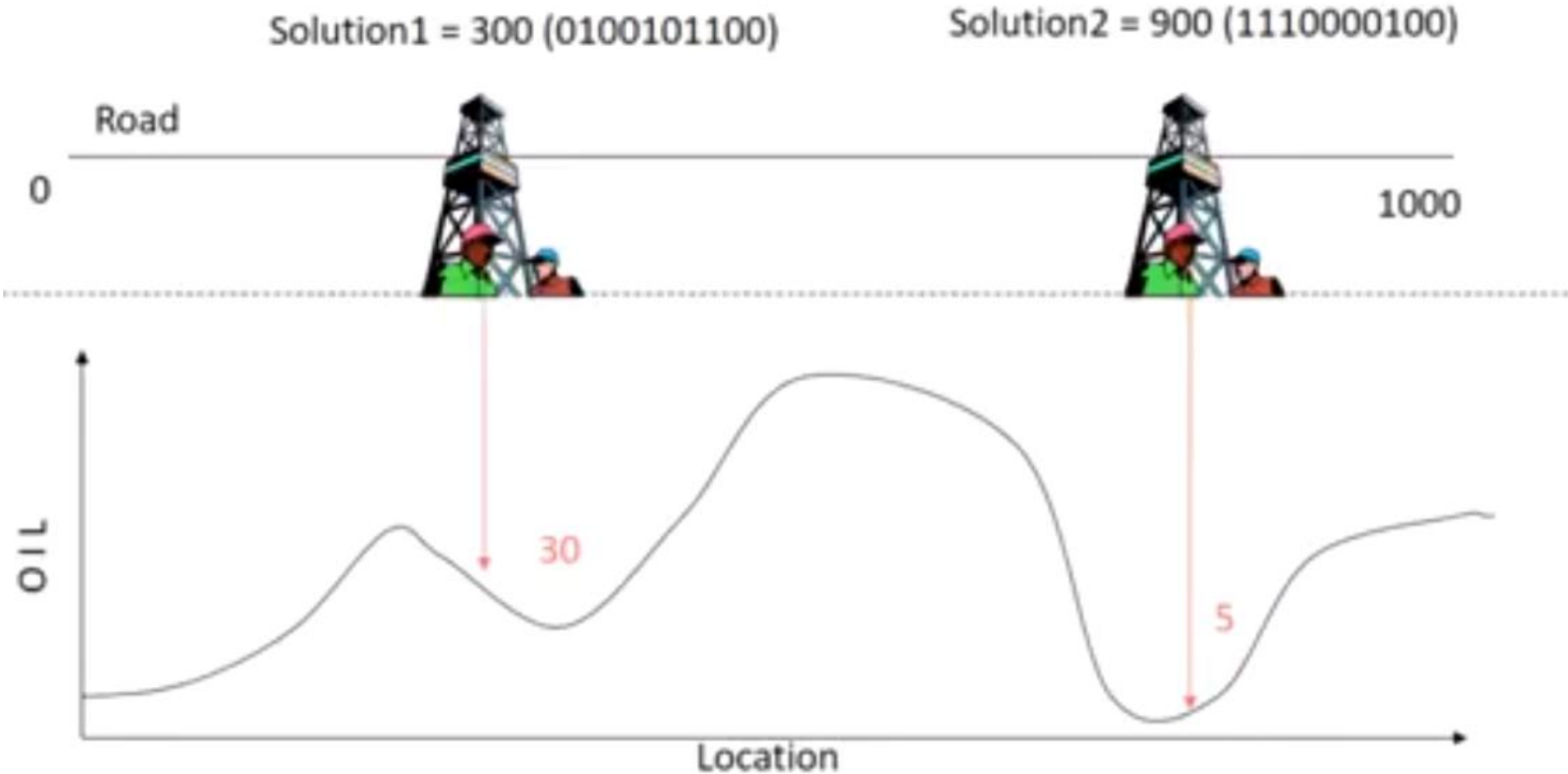
- The set of all possible solutions [0..1000] is called the *search space* or *state space*
- Often GA's code numbers in binary producing a bitstring representing a solution
- In our example we choose 10 bits which is enough to represent 0..1000

0-31 5 bits
0-63 6 bits

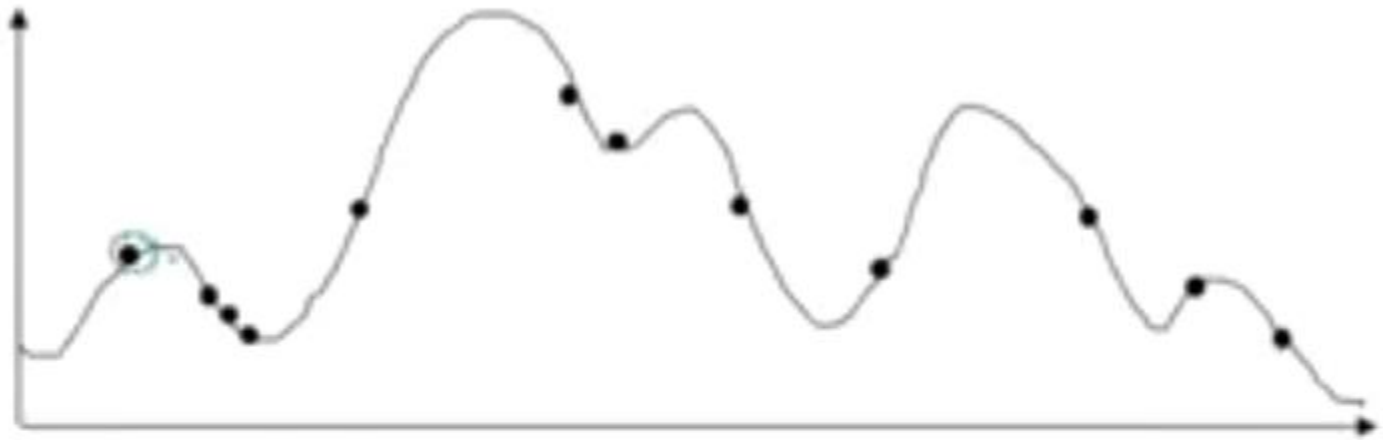
Convert to binary string

	512	256	128	64	32	16	8	4	2	1
900	1	1	1	0	0	0	0	1	0	0
300	0	1	0	0	1	0	1	1	0	0
1023	1	1	1	1	1	1	1	1	1	1

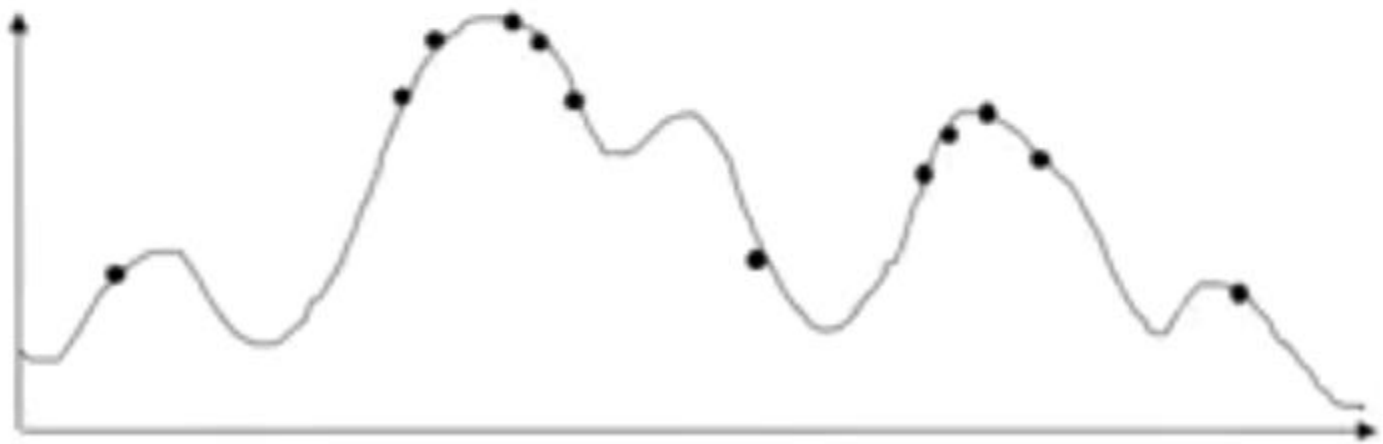
The objective function



Individuals on Curve



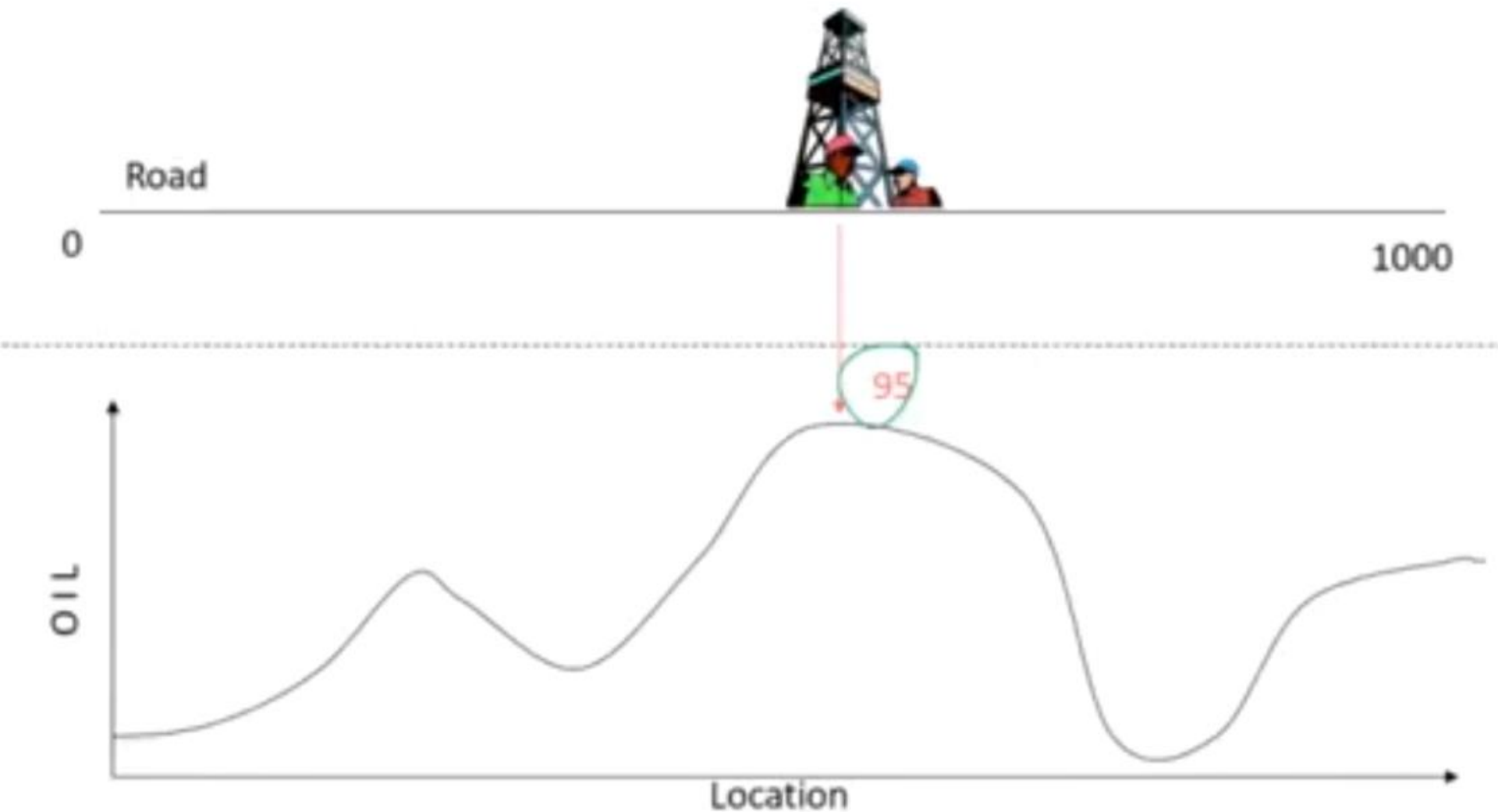
Distribution of Individuals in Generation 0



Distribution of Individuals in Generation N

Optimal Solution

Solution? = 600
(1001011000)



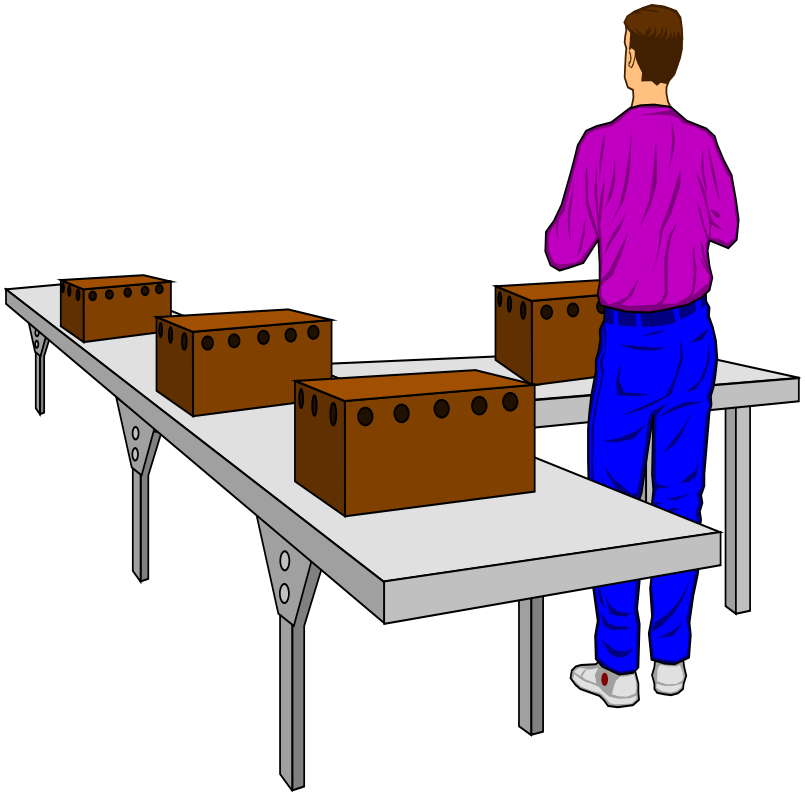
Summary

- Represent possible solutions as a number
- Encoded a number into a binary string
- Ensure that all genotypes correspond to feasible solutions
- Generate a score for each number given a *function* of "how good" each solution is
- Our oil example is really optimisation over a function f(x) where we adapt the parameter x

CONSIDERING THE GA TECHNOLOGY

"Almost eight years ago ... people at Microsoft wrote a program [that] uses some genetic things for finding short code sequences. Windows 2.0 and 3.2, NT, and almost all Microsoft applications products have shipped with pieces of code created by that system."

- Nathan Myhrvold, Microsoft Advanced Technology Group, Wired, September 1995



ISSUES FOR GA PRACTITIONERS

- Choosing basic implementation issues:
  - representation
  - population size, mutation rate, ...
  - selection, deletion policies
  - crossover, mutation operators
- Termination Criteria
- Performance, scalability
- Solution is only as good as the evaluation function (often hardest part)

BENEFITS OF GENETIC ALGORITHMS

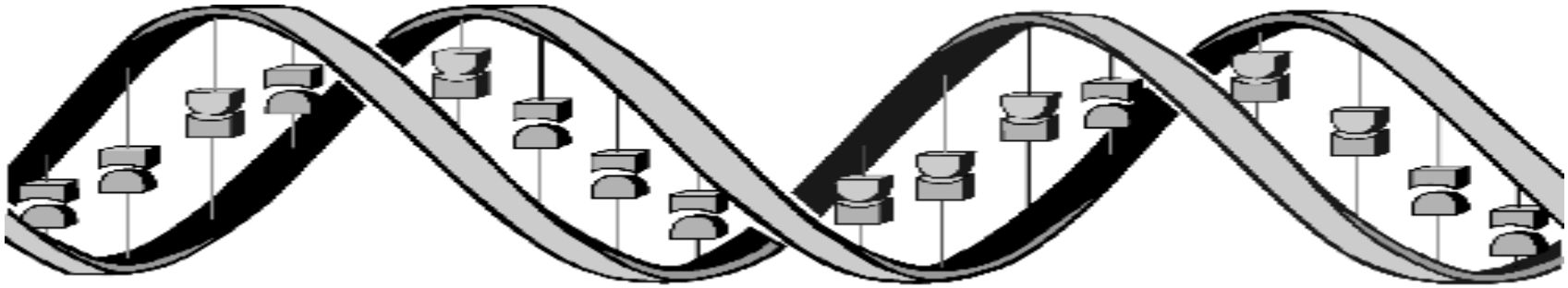
- Concept is easy to understand
- Supports multi-objective optimization
- Good for “noisy” environments
- Always an answer; answer gets better with time
- Inherently parallel; easily distributed
- Many ways to speed up and improve a GA-based application as knowledge about problem domain is gained
- Easy to exploit previous or alternate solutions

WHEN TO USE A GA

- Alternate solutions are too slow or overly complicated.
- Need an exploratory tool to examine new approaches.
- Problem is similar to one that has already been successfully solved by using a GA.
- Want to hybridize with an existing solution.
- Benefits of the GA technology meet key problem requirements

SOME GA APPLICATION TYPES

Domain	Application Types
Control	gas pipeline, pole balancing, missile evasion, pursuit
Design	semiconductor layout, aircraft design, keyboard configuration, communication networks
Scheduling	manufacturing, facility scheduling, resource allocation
Robotics	trajectory planning
Machine Learning	designing neural networks, improving classification algorithms, classifier systems
Signal Processing	filter design
Game Playing	poker, checkers, prisoner's dilemma
Combinatorial Optimization	set covering, travelling salesman, routing, bin packing, graph colouring and partitioning



The End